

## **PHASE 3 — DOCKER & CONTAINERIZATION**

### **DAY 13 — CONTAINER SECURITY**

**Tools:** Docker, Trivy

#### **1. Objectives**

The objectives of Day 13 are:

- Understand minimal container images
- Understand and implement non-root execution
- Understand read-only filesystem concepts
- Understand Linux capabilities
- Understand container resource limits
- Scan container images for vulnerabilities
- Remediate high-value vulnerability findings
- Document accepted-risk decisions

#### **2. Container Security Fundamentals**

Container security focuses on reducing the attack surface of containerized applications and limiting the impact of a compromised container.

Security can be improved by using smaller images, running applications as non-root users, restricting filesystem access, removing unnecessary Linux capabilities, and applying resource limits.

#### **3. Minimal Images**

A minimal container image contains only the software and dependencies required by the application.

Smaller images generally provide:

- A reduced attack surface
- Fewer unnecessary packages
- Fewer potential vulnerabilities
- Smaller image size

- Faster image transfer and deployment

For the application image, the python:3.12-slim base image was used instead of a full Python image.

#### **4. Non-Root Users**

Running an application as a non-root user reduces the privileges available to the application inside the container.

The application image creates a dedicated user:

```
RUN useradd --create-home appuser
```

The Dockerfile then switches to that user:

```
USER appuser
```

Therefore, the application does not run as the root user.

#### **5. Read-Only Filesystem**

A read-only root filesystem prevents processes inside a container from modifying the container's root filesystem.

Docker provides the following runtime option:

```
--read-only
```

Using a read-only filesystem can reduce the ability of an attacker or compromised application to modify files inside the container.

#### **6. Linux Capabilities**

Linux capabilities divide traditionally powerful root privileges into smaller permissions.

Docker containers can run with unnecessary capabilities removed.

The following option was used:

```
--cap-drop=ALL
```

This removes all Linux capabilities from the container unless specific capabilities are explicitly added.

#### **7. Resource Limits**

Resource limits prevent a container from consuming unlimited system resources.

The secured container was configured with:

`--memory=256m`

`--cpus=0.5`

The memory limit restricts the container to 256 MB of memory.

The CPU limit restricts the container to approximately half of one CPU.

Resource limits help reduce the impact of excessive resource consumption.

## **8. Hands-on Practice**

### **8.1 Review the Existing Application Image**

The Day 11 application image was used as the starting point:

`day11-web:1.0`

The existing image was inspected before applying additional security controls.

The image history showed the use of:

WORKDIR

COPY

RUN

USER

EXPOSE

CMD

The image history also confirmed that the application was configured to run using the non-root appuser.

## Screenshot

```
shayan@shayan-VirtualBox:~/day11-dockerfile$ sudo docker history day11-web:1.0
IMAGE          CREATED          CREATED BY          SIZE
COMMENT
96e4ef651901   3 hours ago     CMD ["python" "app.py"]    0B
buildkit.dockerfile.v0
<missing>       3 hours ago     EXPOSE [8000/tcp]         0B
buildkit.dockerfile.v0
<missing>       3 hours ago     USER appuser             0B
buildkit.dockerfile.v0
<missing>       3 hours ago     RUN /bin/sh -c chown -R appuser:appuser /app... 12.
3kB buildkit.dockerfile.v0
<missing>       3 hours ago     COPY app.py . # buildkit  12.
3kB buildkit.dockerfile.v0
<missing>       3 hours ago     WORKDIR /app              8.1
9kB buildkit.dockerfile.v0
<missing>       3 hours ago     RUN /bin/sh -c useradd --create-home appuser... 69.
6kB buildkit.dockerfile.v0
<missing>       6 days ago      CMD ["python3"]           0B
buildkit.dockerfile.v0
<missing>       6 days ago      RUN /bin/sh -c set -eux; for src in idle3 p... 16.
4kB buildkit.dockerfile.v0
<missing>       6 days ago      RUN /bin/sh -c set -eux; savedAptMark="$(a... 41.
4MB buildkit.dockerfile.v0
```

## 8.2 Harden the Container

The existing application image was run with additional runtime security restrictions.

The secured container was configured with:

--read-only

--cap-drop=ALL

--memory=256m

--cpus=0.5

The container was started with:

sudo docker run -d \

--name day13-secure \

--read-only \

--cap-drop=ALL \

--memory=256m \

--cpus=0.5 \

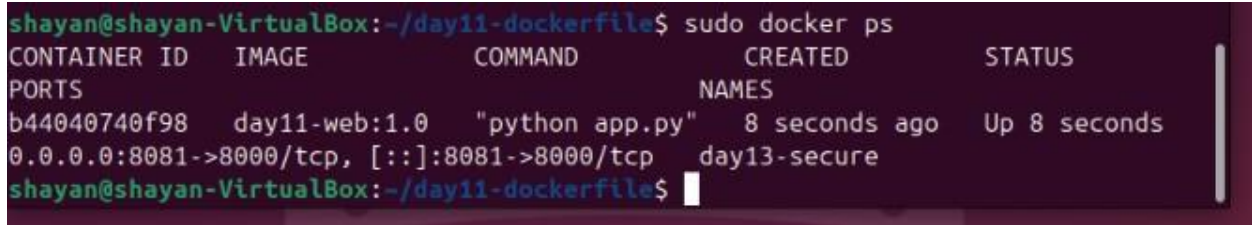
-p 8081:8000 \

day11-web:1.0

These settings provide:

- Read-only root filesystem
- No Linux capabilities
- 256 MB memory limit
- 0.5 CPU limit
- Non-root application execution inherited from the image

### Screenshot



```
shayan@shayan-VirtualBox: ~/day11-dockerfile$ sudo docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS
PORTS         NAMES
b44040740f98   day11-web:1.0  "python app.py"         8 seconds ago  Up 8 seconds
0.0.0.0:8081->8000/tcp, [::]:8081->8000/tcp  day13-secure
shayan@shayan-VirtualBox: ~/day11-dockerfile$
```

### 8.3 Verify Security Configuration

The container configuration was inspected to verify the applied security settings.

The following command was used:

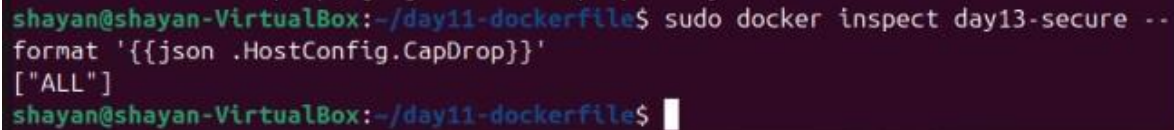
```
sudo docker inspect day13-secure --format 'User={{.Config.User}}
ReadOnly={{.HostConfig.ReadonlyRootfs}} Memory={{.HostConfig.Memory}}
NanoCPUs={{.HostConfig.NanoCpus}}'
```

The container capabilities were also checked using:

```
sudo docker inspect day13-secure --format '{{json .HostConfig.CapDrop}}'
```

The inspection confirmed the configured user, read-only filesystem, resource limits, and dropped capabilities.

## Screenshot



```
shayan@shayan-VirtualBox:~/day11-dockerfile$ sudo docker inspect day13-secure --format '{{json .HostConfig.CapDrop}}'["ALL"]shayan@shayan-VirtualBox:~/day11-dockerfile$
```

## 9. Trivy Vulnerability Scan

Trivy was used to scan the application image for vulnerabilities.

The initial scan command was:

```
sudo trivy image day11-web:1.0
```

A focused vulnerability scan was then performed for HIGH and CRITICAL vulnerabilities:

```
sudo trivy image --scanners vuln --severity HIGH,CRITICAL --format table day11-web:1.0
```

The scan reported:

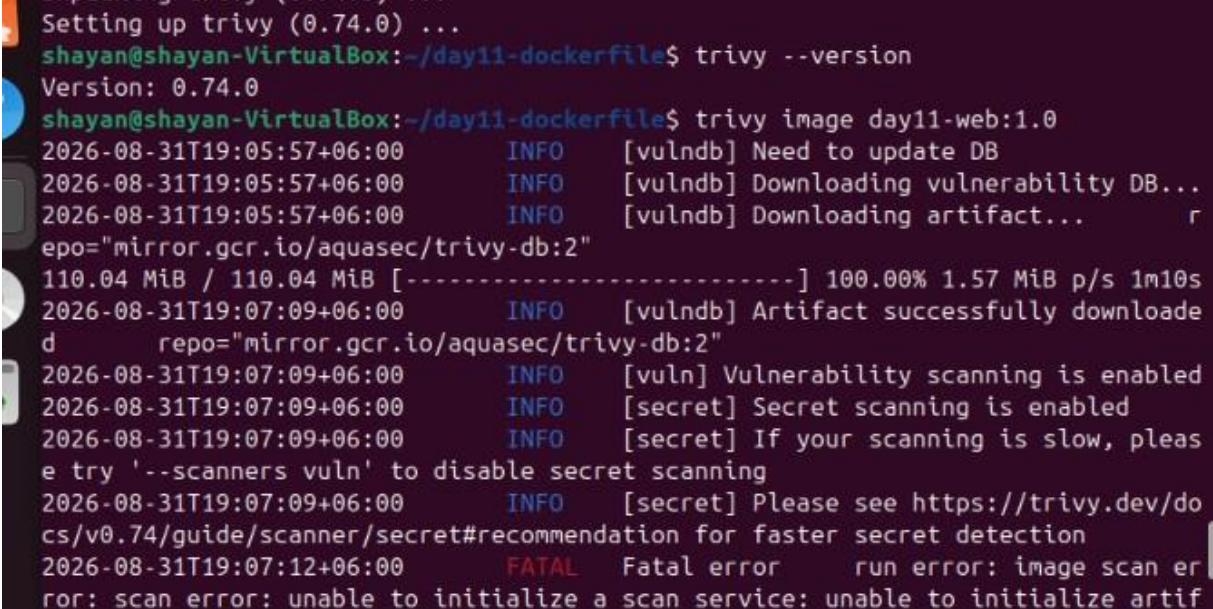
Total: 19

HIGH: 16

CRITICAL: 3

The findings were primarily associated with packages provided by the Debian base image.

## Screenshot



```
Setting up trivy (0.74.0) ...
shayan@shayan-VirtualBox:~/day11-dockerfile$ trivy --version
Version: 0.74.0
shayan@shayan-VirtualBox:~/day11-dockerfile$ trivy image day11-web:1.0
2026-08-31T19:05:57+06:00      INFO      [vulndb] Need to update DB
2026-08-31T19:05:57+06:00      INFO      [vulndb] Downloading vulnerability DB...
2026-08-31T19:05:57+06:00      INFO      [vulndb] Downloading artifact...
repo="mirror.gcr.io/aquasec/trivy-db:2"
110.04 MiB / 110.04 MiB [-----] 100.00% 1.57 MiB p/s 1m10s
2026-08-31T19:07:09+06:00      INFO      [vulndb] Artifact successfully downloaded
repo="mirror.gcr.io/aquasec/trivy-db:2"
2026-08-31T19:07:09+06:00      INFO      [vuln] Vulnerability scanning is enabled
2026-08-31T19:07:09+06:00      INFO      [secret] Secret scanning is enabled
2026-08-31T19:07:09+06:00      INFO      [secret] If your scanning is slow, please try '--scanners vuln' to disable secret scanning
2026-08-31T19:07:09+06:00      INFO      [secret] Please see https://trivy.dev/docs/v0.74/guide/scanner/secret#recommendation for faster secret detection
2026-08-31T19:07:12+06:00      FATAL     Fatal error      run error: image scan error: scan error: unable to initialize a scan service: unable to initialize artif
```

## 10. Vulnerability Remediation Attempt

The Docker image was rebuilt using the python:3.12-slim base image and a new image tag:

```
day13-web:1.0
```

The image was built using:

```
sudo docker build --pull -t day13-web:1.0 .
```

The rebuilt image was then scanned again:

```
sudo trivy image --scanners vuln --severity HIGH,CRITICAL --format table day13-web:1.0
```

The second scan reported:

Total: 19

HIGH: 16

CRITICAL: 3

The rebuild did not reduce the reported HIGH and CRITICAL findings. The remaining findings were associated with packages in the Debian base image rather than application-level Python packages.

## Screenshot

```
shayan@shayan-VirtualBox:~/day11-dockerfile$ sudo trivy image --scanners vuln --severity HIGH,CRITICAL --format table day11-web:1.0
2026-08-31T19:28:25+06:00      INFO      [vuln] Vulnerability scanning is enabled
2026-08-31T19:28:25+06:00      INFO      Detected OS          family="debian" version="13.6"
2026-08-31T19:28:25+06:00      INFO      [debian] Detecting vulnerabilities...  o
s_version="13" pkg_num=87
2026-08-31T19:28:25+06:00      INFO      Number of language-specific files      n
um=1
2026-08-31T19:28:25+06:00      INFO      [python-pkg] Detecting vulnerabilities..
.
2026-08-31T19:28:25+06:00      WARN      Using severities from other vendors for
some vulnerabilities. Read https://trivy.dev/docs/v0.74/guide/scanner/vulnerabil
ity#severity-selection for details.
```

## Report Summary

| Vulnerabilities | Target | Type |
|-----------------|--------|------|
|-----------------|--------|------|

## 11. Accepted-Risk Decision

Because the remaining vulnerabilities were associated with packages inherited from the Debian base image, they were documented as accepted risk for this training exercise.

The application image already uses:

- A slim Python base image
- A dedicated non-root user
- Read-only filesystem at runtime
- Dropped Linux capabilities
- CPU resource limits
- Memory resource limits

The remaining base-image vulnerabilities should be reviewed again when patched versions of the affected packages become available.

The accepted-risk decision was documented in:

SECURITY.md

### Screenshot

A screenshot of a terminal window with a dark purple background. The prompt is 'shayan@shayan-VirtualBox:~/day11-dockerfile\$'. The command 'cat SECURITY.md' has been executed, and the output is displayed in a light blue font. The content of SECURITY.md includes a title, a section header, and the start of a Trivy scan report.

```
shayan@shayan-VirtualBox:~/day11-dockerfile$ cat SECURITY.md
# Day 13 – Container Security Risk Assessment

## Trivy Scan

Image scanned:

day13-web:1.0

Scan scope:
```